

Arquitectura del frontend

El frontend está desarrollado con **React** como Single Page Application (SPA), consumiendo la API REST de Laravel. La aplicación se organiza en:

- Un punto de entrada principal donde se monta React.
- Un sistema de enrutado en el lado del cliente.
- Un conjunto de páginas que representan las distintas áreas de la aplicación (login, gestión de trabajos, prendas, inventario, facturas, notificaciones, calendario, etc.).
- Un módulo centralizado para las llamadas a la API.
- Un contexto de autenticación para gestionar el usuario y su rol.

El objetivo de esta arquitectura es separar claramente la lógica de presentación (componentes), la lógica de negocio (hooks/contexto) y el acceso a datos (API).

Estructura de carpetas

A nivel general, la estructura del frontend sigue este esquema:

text

/src

```
├─ api/      // Funciones para llamar a la API Laravel
├─ components/ // Componentes reutilizables de interfaz
├─ context/   // Contextos globales (por ejemplo, autenticación)
├─ hooks/     // Hooks personalizados
├─ pages/     // Páginas principales (vistas) de la aplicación
├─ router/    // Configuración del enrutado
└─ main.jsx   // Punto de entrada de la aplicación React
```

- En `api/` se agrupan las funciones que llaman a los endpoints `/usuarios`, `/trabajos`, `/prendas`, `/inventario`, `/facturas`, `/notificaciones`, `/eventos`, etc.
- En `pages/` se ubican las pantallas que el usuario ve (por ejemplo, una página de login y páginas relacionadas con cada recurso de la API).
- En `components/` se colocan piezas reutilizables de interfaz (tablas, formularios, cabeceras, etc.).
- En `context/` se define el contexto de autenticación, que guarda el usuario y su rol.
- En `router/` se configura React Router con las rutas públicas y protegidas.

Organización de componentes

Los componentes se organizan en dos niveles:

- **Páginas (pages):** son los componentes que representan una vista completa. Cada página suele corresponderse con una funcionalidad principal de la API (por ejemplo, una página para listar trabajos, otra para gestionar inventario, otra para ver facturas, etc.), además de una página de login.
- **Componentes reutilizables (components):** son piezas más pequeñas que se usan dentro de las páginas, como formularios, listados, tablas o elementos de navegación.

Las páginas se encargan de:

- Llamar a la API (directamente o a través de hooks).
- Gestionar el estado local de esa vista (filtros, formularios, etc.).
- Renderizar componentes reutilizables pasándoles los datos como props.

Gestión de estado (Context, hooks, etc.)

La gestión del estado global se centra principalmente en la **autenticación** y el **rol del usuario**:

- Se utiliza un **Context de autenticación** para almacenar:
 - El usuario autenticado.
 - El token de sesión (proporcionado por Laravel Sanctum).
 - El rol del usuario (admin, empleado, cliente).
- Sobre ese contexto se construye un **hook personalizado** (por ejemplo, useAuth) que expone funciones como:
 - iniciar sesión (login),
 - cerrar sesión (logout),
 - comprobar si el usuario está autenticado,
 - comprobar el rol del usuario.

El resto del estado (datos de trabajos, prendas, inventario, etc.) se gestiona normalmente a nivel de cada página con useState y useEffect, o con pequeños hooks específicos si se reutiliza lógica.

Sistema de enrutado

El enrutado se implementa con **React Router**.

A nivel conceptual, se distinguen tres tipos de rutas:

1. Rutas públicas

- Por ejemplo, la ruta de login.

2. Rutas privadas (requieren estar autenticado)

- Por ejemplo, rutas que muestran trabajos, facturas, notificaciones, etc.

3. Rutas protegidas por rol

- Se alinean con los middlewares del backend, por ejemplo:
 - Rutas solo para admin (gestión de usuarios, borrado de recursos).
 - Rutas para admin y empleado (gestión de prendas, inventario, etc.).
 - Rutas para empleado y cliente (creación de trabajos, etc.).

En el frontend, esto se resuelve con componentes de ruta protegida que:

- Comprueban si hay usuario autenticado (token válido).
- Comprueban si el rol del usuario está dentro de los roles permitidos para esa ruta.

Si no se cumple, se redirige al login o a una página de acceso denegado.